

ROS2 Spring Workshop

Lab 6: The Weed Sprayer (Computer Vision & OpenCV)

Project Context: The “Weed Sprayer”

In **Phase 2** of your Final Project, your robot must identify red weed markers on the field, drive over them, and stop to spray. The robot won’t use the God Node to find the markers; it must use its camera to hunt for targets. **This lab builds that vision brain.** Our markers will be **red balls**. The choice of a spherical target is not accidental; more on this in Part 4.

Objective

In this lab, we give the robot eyes. To make development faster, we will run the camera and the code **directly on your laptop**.

You will learn:

- **OpenCV:** The common library for computer vision.
- **HSV Colour Space:** How to detect “red” regardless of lighting.
- **Shape Analysis:** Using math to distinguish a red circle from a red shoe.

Pre-Lab Concept Primer: The Robot Eye

1. What is an Image? (The Spreadsheet)

To us, a picture is a visual scene. To a computer, a picture is just a **spreadsheet**.

- A standard HD image (1920×1080) is just a grid of 2,073,600 pixels.
- Each pixel is a container holding 3 numbers in the range 0–255: Blue, Green, Red. (Yes, BGR not RGB. This is an OpenCV convention that surprises everyone once; you will see it again in Part 4.)
- **Computer vision** is simply performing math on this giant spreadsheet.

2. The “Colour Space” Trap (RGB vs. HSV)

This is the most common mistake in robotics.

- **RGB (Red-Green-Blue):** This is how *screens* work. It mixes light. The camera measures how much R, G, B light is bouncing off an object, and that bounced light is the product of the object’s surface AND the lighting in the room. When lighting changes, all three channels move at once: a red apple in full sun might read (230, 40, 30), but in shadow it drops to roughly (95, 15, 12). Same apple, every number different. Any absolute threshold like “ $R > 200$ ” breaks based on the cloud, not the apple.
- **HSV (Hue-Saturation-Value):** This is how *humans* describe colour. By looking only at the **Hue (H)** value, we can tell the robot “look for Hue ≈ 0 (red)” regardless of how bright or dark the room is.

3. The “Red Hue Wraparound (Gotcha!)”

Red is weird in HSV. OpenCV compresses the full 360 colour wheel into Hue values 0–179 (so it fits in a single byte). Red sits at BOTH ends of that range: near $H = 0$ and near $H = 179$. If your calibration shows some pixels near $H = 5$ and some near $H = 175$, those are both red, just on opposite sides of the wheel.

The fix: for red specifically, we use TWO masks, one for each red range, and OR them together with `cv2.bitwise_or()`. A single range like $[0, 10]$ often *appears* to work under controlled lighting, then breaks the moment the white balance shifts or the room dims. Use two ranges from the start.

4. Shape Analysis (Circularity)

We use math to distinguish specific shapes from random noise. The **circularity** formula compares area to perimeter:

$$Circularity = \frac{4\pi \cdot Area}{Perimeter^2}$$

The math: A circle has the most efficient area-to-perimeter ratio in geometry, which is what the formula measures.

Worked example: A 30×30 pixel square has area = 900 and perimeter = 120. Plugging in:

$$Circularity = \frac{4\pi \cdot 900}{120^2} = \frac{11,310}{14,400} \approx 0.785$$

A circle drawn at the same resolution gives ≈ 0.95 due to pixelation artifacts on the jagged edges.

Reference values:

- Perfect circle: 1.0 (unachievable in pixels)
- Pixelated circle, clean image: ~ 0.90 to 0.98
- Real webcam capture of a textured ball: ~ 0.75 to 0.90 (shadows, JPEG compression, and shiny spots all knock the value down)
- Square: ≈ 0.785
- Weird blob (shoe/leaf): < 0.5

This is why we use 0.7 rather than 0.8 as our threshold: real images leave less margin than theory suggests.

The Two Filters: Size First

After `findContours()` finds white blobs in the mask, the detector checks each blob with two filters.

Filter 1: Size

```
if area > 500 and perimeter > 0:
```

The area is the number of pixels inside the blob. Tiny blobs are usually noise, reflections, or small red objects in the background. The value 500 means the blob must be large enough to matter.

Filter 2: Shape

```
circularity = (4 * math.pi * area) / (perimeter * perimeter)
```

```
if circularity > 0.7:
```

Circularity checks whether the blob is roughly circular. A perfect circle has circularity 1.0. A messy shape has a lower value. We use 0.7 because real webcam images have shadows, jagged edges, and imperfect masks.

The order matters: the detector checks size first because there is no point calculating circularity for tiny noise specks.

Part 1: Laptop Hardware Setup

We will install the drivers directly on your laptop (or VM). These tools allow ROS to communicate with your USB hardware and visualize the data.

1. **Install the camera driver & visualizer:** *Note: We install the camera driver and the RQT image viewer together to save time.*

```
1 sudo apt-get update
2 sudo apt-get install ros-humble-usb-cam ros-humble-rqt-image-view
3
```

2. **Install OpenCV (Python):** *Note: We use the system package manager to ensure compatibility with the ROS 2 environment.*

```
1 sudo apt-get install python3-opencv
2
```

OpenCV Commands Used in This Lab

You do not need to memorize these OpenCV functions, but you should know what each one does in this lab.

- `cv2.cvtColor()` converts the camera image from BGR colour format to HSV colour format.
- `cv2.inRange()` creates a black-and-white mask. Pixels inside the HSV range become white, and pixels outside the range become black.
- `cv2.bitwise_or()` combines the two red masks into one final red mask.
- `cv2.erode()` shrinks white regions in the mask. This helps remove tiny white speckles.
- `cv2.dilate()` grows white regions in the mask. After erosion removes speckles, dilation grows the main ball blob back.
- `cv2.findContours()` finds the outlines of white blobs in the mask.
- `cv2.contourArea()` measures the area inside a contour.
- `cv2.arcLength()` measures the perimeter of a contour.
- `cv2.drawContours()` draws the detected contour on the debug image.
- `cv2.boundingRect()` finds a rectangle around a contour. In Part 7, we use this to calculate the weed centre.
- `cv2.rectangle()` draws the treatment zone box.
- `cv2.circle()` draws a dot at the weed centre.

Helpful OpenCV references:

- HSV thresholding with `inRange()`: https://docs.opencv.org/4.x/da/d97/tutorial_threshold_inRange.html
- Contours with `findContours()` and `drawContours()`: https://docs.opencv.org/4.x/d4/d73/tutorial_py_contours_begin.html
- Morphological operations such as `erode()` and `dilate()`: https://opencv24-python-tutorials.readthedocs.io/en/latest/py_tutorials/py_imgproc/py_morphological_ops/py_morphological_ops.html

3. **The “Hardware Handshake” (Windows Users):** If you are using Docker on Windows, you must manually “pass” the USB device from Windows to the Linux kernel.

- Open PowerShell (Admin) and run: `usbipd list`
- Bind and attach your camera’s Bus ID (e.g., 1-13):

```
1 usbipd bind --busid 1-13
2 usbipd attach --wsl --busid 1-13
3
```

Heads-Up: Bandwidth & MJPEG

Virtual USB connections (Docker on Windows, especially) have limited bandwidth. Raw video formats like YUYV are huge and cause **Select Timeout** errors. To prevent this, we launch the camera driver with `pixel_format:=mjpeg2rgb` in Part 6. If you see timeout errors there, this is why.

Part 2: Package Creation

1. Navigate to your source folder:

```
1 cd /workspaces/agtech_ros2/src
2
```

2. Create the package:

```
1 ros2 pkg create --build-type ament_python lab6_vision --dependencies rclpy
  sensor_msgs std_msgs cv_bridge
2
```

3. **Import resources:** Copy the starter files from the resources folder into your new package:

```
1 # Navigate to the Python folder inside your new package
2 cd /workspaces/agtech_ros2/src/lab6_vision/lab6_vision
3
4 # Copy the starter files
5 cp /workspaces/agtech_ros2/lab_resources/lab6/*.py .
6
```

Part 3: The Calibrator Tool

The biggest frustration in computer vision is guessing the numbers. *“Is this red 170? Or 175?”*

Task: Run the `colour_calibrator.py` script. Notice that we are running this directly as a standard Python script, **not** as a ROS 2 node.

```
1 python3 colour_calibrator.py
```

Why Python Directly

The calibrator uses OpenCV’s direct webcam access (`cv2.VideoCapture(0)`) instead of subscribing to a ROS topic. This is faster for calibration; no ROS nodes to launch, no topics to remap. Your weed detector (Part 4) will use ROS since it has to integrate with the rest of the robot.

Hold your red ball in front of the camera. Adjust the sliders until the ball is **white** in the mask and the background is **black**.

When you press **p**, the calibrator prints two red HSV ranges. Copy all four arrays into `weed_detector.py`: `lower_red1`, `upper_red1`, `lower_red2`, and `upper_red2`.

Crucial Step: Window Focus

To use the **p** (print) or **q** (quit) keyboard commands, **you must click on one of the OpenCV pop-up windows** (e.g., ‘Camera Feed’ or ‘Colour Tuner’) to bring it into focus. If your VS Code editor or terminal is actively selected, the script will completely ignore your key presses!

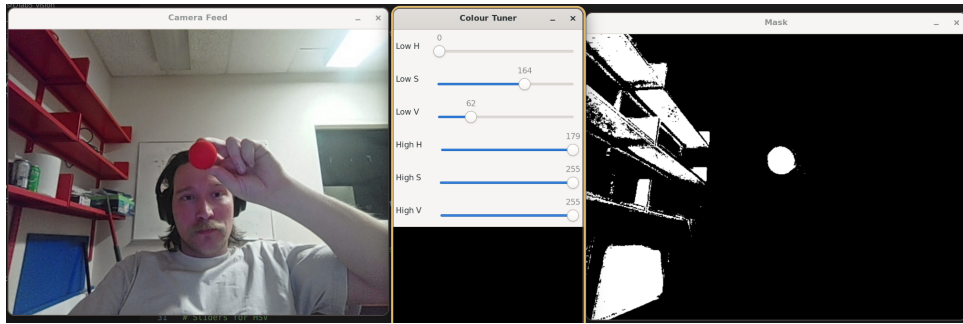


Figure 1: The Colour Calibrator: Tuning the sliders to isolate the marker (white) from the background (black).

Debugging Tip: The Mask Is the Real Detector

The robot does not actually “see” a red ball. It sees a black-and-white mask. White pixels are pixels that passed the HSV test; black pixels were rejected.

If detection is flickering, always inspect the mask first:

- If the ball is a solid white circle and the background is black, your HSV values are good.
- If the ball is broken into pieces, lower your saturation/value thresholds.
- If the table, hands, face, or background also appear white, raise your saturation threshold or improve the lighting/background.

OpenCV stores colour images in BGR order by default, meaning the channels are Blue, Green, Red instead of Red, Green, Blue. That is why we use `cv2.COLOR_BGR2HSV`. HSV is better for colour detection because it separates colour type from brightness. Hue tells us the basic colour, Saturation tells us how strong that colour is, and Value tells us how bright the pixel is.

From Mask to Detection: The Filter Pipeline

Once the robot has a black-and-white mask, it still needs to decide which white blobs are actually weed markers. The detector uses a simple filter pipeline:

1. **Colour Filter:** `cv2.inRange()` keeps only pixels that match our red HSV ranges.
2. **Mask Cleanup:** `erode()` and `dilate()` remove tiny white speckles from the mask.
3. **Contour Detection:** `findContours()` finds the outline of each white blob.
4. **Size Filter:** Very small blobs are ignored because they are usually noise or reflections.
5. **Shape Filter:** Circularity checks whether the blob is roughly circle-shaped.

This is a common computer vision pattern: first isolate the pixels you care about, then clean the result, then measure the blobs.

Part 4: The “Weed Detector” Node

This node listens to the camera, filters for red, and uses math to find only circular blobs.

Geometry Insight: Why Red Balls?

A ball (sphere) has a property mathematicians call **point symmetry**: every axis through the centre is a symmetry axis. One practical consequence is that **the silhouette of a sphere projected onto a 2D image is always a circle**, no matter what angle the camera is looking from.

A sphere viewed from above? Circle. From 45 degrees? Circle. From the side? Circle (just smaller).

Compare that to a flat disc: from directly above it's a circle, but from any other angle it becomes an ellipse with circularity dropping as low as 0.5. A camera mounted on a robot looks at the floor from an angle, so flat discs would be a perspective nightmare.

By choosing red balls as our markers, we sidestep the perspective problem entirely. In clean images, a high circularity threshold such as > 0.8 works well. In classroom lighting with webcams, reflections, and compression artifacts, we will use a slightly more forgiving threshold of > 0.7 .

Task: Open `weed_detector.py` in VS Code and fill in the blanks.

```
1 #!/usr/bin/env python3
2 """
3 Lab 6: Weed Detector - red ball detection using HSV + circularity.
4
5 Subscribes:
6     /image_raw           sensor_msgs/Image   - Camera feed
7
8 Publishes:
9     /weed_detector/debug  sensor_msgs/Image   - Frame with weeds outlined
10    /weed_detector/mask    sensor_msgs/Image   - Black/white HSV mask
11    /tractor/spray         std_msgs/Bool       - True when weed detected
12 """
13
14 import rclpy
15 from rclpy.node import Node
16 from sensor_msgs.msg import Image
17 from std_msgs.msg import Bool
18 from cv_bridge import CvBridge
19 import cv2
20 import numpy as np
21 import math
22
23
24 class WeedDetector(Node):
25     def __init__(self):
26         super().__init__('weed_detector')
27
28         # 1. Subscribe to the raw camera feed.
29         # The usb_cam driver publishes to /image_raw by default.
30         self.sub = self.create_subscription(
31             Image, '/image_raw', self.image_callback, 10
32         )
33
34         # 2. Publishers.
35         # /weed_detector/debug shows the camera image with green outlines.
36         self.pub_debug = self.create_publisher(
37             Image, '/weed_detector/debug', 10
38         )
39
40         # /weed_detector/mask shows the black/white HSV mask.
41         # White pixels are pixels the detector thinks are red.
42         self.pub_mask = self.create_publisher(
43             Image, '/weed_detector/mask', 10
44         )
45
46         # /tractor/spray publishes True when a weed marker is detected.
47         self.pub_spray = self.create_publisher(
48             Bool, '/tractor/spray', 10
49         )
50
51         self.bridge = CvBridge()
52         self.get_logger().info("Weed detector active. Waiting for images...")
```

```

53
54 def image_callback(self, msg):
55     try:
56         # Convert the ROS Image message into an OpenCV image.
57         # "bgr8" is OpenCV's normal colour format.
58         cv_image = self.bridge.imgmsg_to_cv2(msg, "bgr8")
59     except Exception as e:
60         # Log the error so students can see why the callback failed.
61         self.get_logger().error(f"CvBridge conversion failed: {e}")
62         return
63
64     # 3. Convert from BGR to HSV.
65     # HSV makes colour filtering easier because Hue represents the colour,
66     # while Saturation and Value represent colour intensity and brightness.
67     hsv_image = cv2.cvtColor(cv_image, cv2.COLOR_BGR2HSV)
68
69     # 4. Create the red mask.
70     # Red wraps around the HSV hue wheel in OpenCV.
71     # This means red appears near H = 0 and also near H = 179.
72     # Therefore, we create two masks and combine them.
73
74     # Range 1: lower end of red hue range.
75     # Paste the first range printed by colour_calibrator.py here.
76     lower_red1 = np.array([<LOW_H1>, <LOW_S>, <LOW_V>])
77     upper_red1 = np.array([<HIGH_H1>, <HIGH_S>, <HIGH_V>])
78
79     # Range 2: upper end of red hue range.
80     # Paste the second range printed by colour_calibrator.py here.
81     lower_red2 = np.array([<LOW_H2>, <LOW_S>, <LOW_V>])
82     upper_red2 = np.array([<HIGH_H2>, <HIGH_S>, <HIGH_V>])
83
84     mask1 = cv2.inRange(hsv_image, lower_red1, upper_red1)
85     mask2 = cv2.inRange(hsv_image, lower_red2, upper_red2)
86
87     # Combine both red masks into one final mask.
88     mask = cv2.bitwise_or(mask1, mask2)
89
90     # 5. Clean up the mask before finding contours.
91     # Small reflections, camera noise, and compression artifacts can create
92     # tiny white speckles. Erode removes small white dots, and dilate grows
93     # the remaining white regions back to roughly their original size.
94     kernel = np.ones((5, 5), np.uint8)
95     mask = cv2.erode(mask, kernel, iterations=1)
96     mask = cv2.dilate(mask, kernel, iterations=1)
97
98     # Publish the mask so it can be viewed in rqt_image_view.
99     self.pub_mask.publish(self.bridge.cv2_to_imgmsg(mask, "mono8"))
100
101     # 6. Find contours.
102     # A contour is the outline of a white blob in the mask.
103     # RETR_EXTERNAL keeps only the outermost contours, which is enough
104     # for this lab because we only care about separate red blobs.
105     contours, _ = cv2.findContours(
106         mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
107     )
108
109     found_weed = False
110
111     for contour in contours:
112         area = cv2.contourArea(contour)
113         perimeter = cv2.arcLength(contour, True)
114
115         # Filter 1: Size.
116         # Ignore tiny blobs under 500 pixels. These are usually noise,
117         # reflections, or small red objects in the background.
118         if area > 500 and perimeter > 0:
119
120             # Filter 2: Shape.
121             # Circularity compares the blob's area to its perimeter.
122             #
123             # Formula:
124             # circularity = 4 * pi * Area / Perimeter^2
125             #

```

```

126         # A perfect circle has circularity = 1.0.
127         # In real camera images, a red ball may score lower because of
128         # shadows, pixelated edges, blur, or imperfect masking.
129         circularity = (4 * math.pi * area) / (perimeter * perimeter)
130
131         # Since our markers are red balls, their 2D silhouette should
132         # be roughly circular. A threshold of 0.7 is forgiving enough
133         # for classroom lighting and webcam noise.
134         if circularity > 0.7:
135             found_weed = True
136
137         # Draw a green outline around the detected weed marker.
138         cv2.drawContours(cv_image, [contour], -1, (0, 255, 0), 3)
139
140         # 7. Publish spray trigger.
141         # True means a red circular marker was found in this frame.
142         spray_msg = Bool()
143         spray_msg.data = found_weed
144         self.pub_spray.publish(spray_msg)
145
146         # 8. Publish debug image.
147         # This image is the original camera feed with green outlines drawn on it.
148         self.pub_debug.publish(self.bridge.cv2_to_imgmsg(cv_image, "bgr8"))
149
150
151 def main(args=None):
152     rclpy.init(args=args)
153     node = WeedDetector()
154
155     try:
156         rclpy.spin(node)
157     except KeyboardInterrupt:
158         pass
159     finally:
160         # Safety shutdown:
161         # Publish one final False so the sprayer does not remain active
162         # after the node is stopped.
163         stop_msg = Bool()
164         stop_msg.data = False
165         node.pub_spray.publish(stop_msg)
166
167         node.destroy_node()
168         rclpy.shutdown()
169
170
171 if __name__ == '__main__':
172     main()

```

Listing 1: weed_detector.py (Fill in the TODOs)

Code Walkthrough: What Each Block Does

This script has a lot of moving parts, so do not try to memorize every line. Instead, focus on the job of each block.

- **Imports:** The script imports the ROS 2 tools, image message types, OpenCV, NumPy, and math. The important new piece is `CvBridge`, which converts between ROS image messages and OpenCV images.
- **Node Setup:** The `WeedDetector` class creates one subscriber and three publishers. The subscriber listens to `/image_raw`. The publishers send the annotated debug image to `/weed_detector/debug`, the black-and-white mask to `/weed_detector/mask`, and the spray decision to `/tractor/spray`.
- **Image Conversion:** The camera image arrives as a ROS `Image` message. OpenCV cannot work with that message directly, so `CvBridge` converts it into a normal OpenCV image using `imgmsg.to_cv2(msg, "bgr8")`.
- **BGR to HSV:** OpenCV stores colour images in BGR order by default. The detector converts the image to HSV because HSV makes colour filtering easier. Hue describes the colour, while Saturation and Value describe colour strength and brightness.

- **Two Red Masks:** Red appears at both ends of the OpenCV Hue scale, near $H = 0$ and near $H = 179$. The detector creates one mask for each red range, then combines them using `cv2.bitwise_or()`.
- **Mask Cleanup:** Real camera masks often contain tiny white speckles from reflections, compression artifacts, or lighting. The detector uses `erode()` to shrink white regions and remove tiny speckles, then `dilate()` to grow the main ball blob back.
- **Mask Publishing:** The cleaned mask is published to `/weed_detector/mask`. This lets you inspect the exact black-and-white image that the detector is using. If the mask is bad, the detection will be bad.
- **Contour Detection:** `findContours()` finds the outlines of white blobs in the mask. The detector uses `RETR_EXTERNAL` because we only care about the outer boundary of each red blob.
- **Size Filter:** The detector ignores blobs with area below 500 pixels. Small blobs are usually noise, reflections, or tiny red objects in the background.
- **Shape Filter:** For each large enough blob, the detector calculates circularity:

$$Circularity = \frac{4\pi \cdot Area}{Perimeter^2}$$

A perfect circle has circularity 1.0. In real camera images, the value may be lower because of shadows, jagged mask edges, or blur. In this lab, a blob passes the shape filter if its circularity is greater than 0.7.

- **Drawing the Debug Image:** If a blob passes both filters, the detector draws a green outline around it on the OpenCV image. This annotated image is published to `/weed_detector/debug`.
- **Publishing the Spray Decision:** The detector stores the result in a Python Boolean called `found_weed`. It then wraps that value in a ROS `Bool` message and publishes it to `/tractor/spray`. This is how the rest of the robot can react to the vision system.
- **Safe Shutdown:** When the node shuts down, it publishes one final `False` message to `/tractor/spray`. This prevents the sprayer from staying on because of an old `True` message.

Real-World CV: False Positives

Your detector fires on *anything* red and circular. In the Triathlon arena this is fine because the instructor controls what's on the floor. But in a real farm you would get false positives from:

- Discarded red drink cans (red AND roughly circular from above)
- Wet red leaves flattened by rain
- Ladybugs at close range
- Red tape or paint on tools left in the field

How would you reduce false positives? You do not need to implement this just think about it. Ideas:

- **Size bounds:** Reject contours that are too large (a soda can) or too small (a ladybug).
- **Temporal filtering:** Require the detection to persist for N frames before firing, to reject single-frame flickers.
- **Aspect ratio check:** The bounding box of a real sphere is close to square (1:1). Reject contours with wildly elongated bounding boxes.
- **Context:** Use the robot's GPS; only trust detections that happen inside known crop rows.

Real agricultural CV engineers spend more time on false-positive rejection than on the initial detection.

Mask Cleanup: How Erode and Dilate Work

Real camera masks are noisy. Reflections, compression artifacts, and shiny surfaces create tiny white speckles that can confuse contour detection. Before finding contours, we clean the mask:

- `erode()` shrinks white regions. Tiny speckles disappear completely; the main ball blob just gets a bit smaller.
- `dilate()` grows white regions. After erosion removes the speckles, dilation grows the ball blob back toward its original size. The speckles stay gone because they no longer exist to grow back from.

In this lab, we use them together:

```
kernel = np.ones((5, 5), np.uint8)
mask = cv2.erode(mask, kernel, iterations=1)
mask = cv2.dilate(mask, kernel, iterations=1)
```

What is a kernel? A small grid (5×5 pixels in our case) that OpenCV slides across the mask. At each pixel, it examines the 25 neighbours around it and applies the rule (erode: “if any neighbour is black, turn this pixel black”; dilate: “if any neighbour is white, turn this pixel white”). A larger kernel removes more noise but can damage small objects too. For our red ball markers, 5×5 is a good starting point.

Part 5: Registration & Build

1. Open `setup.py` and add the entry point:

```
1 'console_scripts': [  
2     'weed_detector = lab6_vision.weed_detector:main',  
3 ],  
4
```

2. Build and source:

```
1 cd /workspaces/agtech_ros2  
2 colcon build --symlink-install  
3 source install/setup.bash  
4
```

Part 6: Running the Full System

Robotics requires multiple processes running at once. Open **three separate terminals**.

1. **Terminal 1: The Camera Driver.** This turns on the hardware. We use `mjpeg2rgb` to prevent USB timeouts (the bandwidth issue from Part 1).

```
1 ros2 run usb_cam usb_cam_node_exe --ros-args -p pixel_format:"mjpeg2rgb"  
2
```

Reminder: Check Your Exposure!

If your weed detector struggles to find the target, your camera’s auto-exposure might have gotten stuck when the node launched. If the image looks completely dark or blown out, stop this driver (`Ctrl+C`) and run these commands to force auto-exposure and reset brightness to the middle, then restart the driver:

```
sudo apt install v4l-utils          # if not installed  
v4l2-ctl -d /dev/video0 -c auto_exposure=3  
v4l2-ctl -d /dev/video0 -c brightness=128
```

2. **Terminal 2: Your Vision Node.** This runs your logic.

```

1 source install/setup.bash
2 ros2 run lab6_vision weed_detector
3

```

3. **Terminal 3: Visualization.** Open the viewer to see what your robot sees.

```

1 ros2 run rqt_image_view rqt_image_view
2

```

Sanity Check: Does the ROS image match the Python image?

In `rqt_image_view`, check:

- `/image_raw` for the raw camera feed.
- `/weed_detector/mask` for the black-and-white HSV mask.
- `/weed_detector/debug` for the final annotated detection image.

The mask should show the red ball as a clean white blob. If other objects also appear white, your HSV thresholds are too broad or the background is reflecting red light.

You might wonder: “*Will the colours look different here than they did in the Python calibrator?*”

We designed the Python script to match the ROS node closely. The script requests the MJPG video codec at 640x480, which mirrors the `mjpeg2rgb` parameter you passed to `usb_cam`. The colours should match closely. If the ROS mask looks terrible compared to your calibration mask, check your lighting and exposure (orange box above).

Part 7: Programming Challenge: Precision Spraying

Right now, your node publishes `True` if a weed is *anywhere* in the frame. But spray chemicals are expensive and overuse is harmful to the environment! If the weed is at the very edge of the camera’s vision, the nozzle won’t hit it.

Your challenge: Upgrade your `weed_detector.py` so it only triggers the spray when the weed is directly in the **centre** of the camera’s view (the “treatment zone”). If the weed is visible but off-centre, the node should calculate its coordinates to help the tractor navigate toward it (or the nozzle!). I have provided you with a template in the lab resource folder called `precision_spray.py`. Refer to the text box of OpenCV commands at the beginning of the lab for extra explanations.

The Specifications

Modify your Python script to implement the following logic:

1. **Find the Centre of the Weed:** Instead of just finding the contour, find its bounding box and calculate the centre pixel (C_x, C_y).
2. **Define the Treatment Zone:** Calculate the exact centre of your camera’s overall image frame. Define a “targeting box” that is 100×100 pixels around that centre point.
3. **Draw the HUD (Heads-Up Display):** Use `cv2.rectangle` to draw your 100×100 treatment zone box on the screen in blue. Draw a small dot or cross at the weed’s (C_x, C_y) coordinate.
4. **The Decision Logic:**
 - **If weed is OUTSIDE the box:** Do NOT spray (`False`). Log an info message: “*Weed detected at (X, Y). Navigating...*”
 - **If weed is INSIDE the box:** Trigger the spray (`True`). Log a warning message: “*WEED CENTRED. SPRAYING!*”

Developer Hint: Finding the Centre

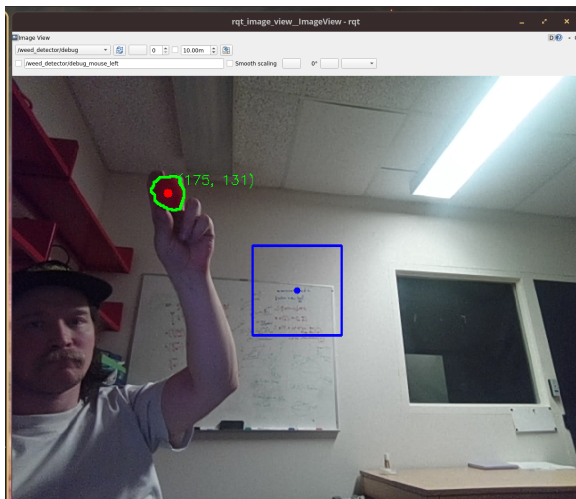
Once you have a valid contour that passed your size and circularity filters, you can easily find its (X, Y) centre using a bounding rectangle:

```
# Get the bounding box of the contour
x, y, w, h = cv2.boundingRect(contour)

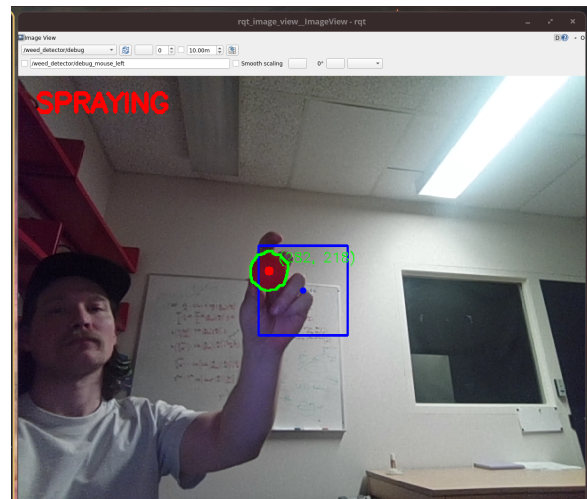
# Calculate the centre point
center_x = int(x + (w / 2))
center_y = int(y + (h / 2))
```

Connecting the Dots to Lab 5

Notice how you are logging the (X, Y) pixel coordinates when the weed is off-centre? In a fully integrated God Node system, you would take those pixel coordinates, run them through the camera matrix and `camera_info` transforms you calibrated in **Lab 5**, and convert them into real-world (X, Y) metres. You could then publish those spatial coordinates to a navigation stack to make the tractor drive over the weed automatically!



Weed detected, but outside the treatment zone. The sprayer remains off.



Weed centred inside the treatment zone. The sprayer is triggered.

Figure 2: Precision spraying logic. The robot only sprays when the detected weed marker is inside the centre treatment zone.

Deliverables

Submit a single PDF containing:

1. **Calibration:** Screenshot of `colour_calibrator.py` isolating red.
2. **Detection:** Screenshot of `rqt_image_view` showing `/weed_detector/debug` with a green contour around a red ball and `/weed_detector/mask` with the ball as a white blob.
3. **Logic:** Screenshot of terminal echoing `/tractor/spray: True` when a ball is in view.
4. **Precision Spray:** A screenshot of your debug image showing the blue treatment zone box, the weed contour, and your terminal logging "WEED CENTRED. SPRAYING!"
5. **Reflection (3–5 sentences):** From the False Positives box in Part 4, pick one rejection strategy (size bounds, temporal filtering, aspect ratio, or GPS context) and explain how you would implement it. Which other fruits/vegetables would be problematic for this detector in a real field?